week5 예습과제

다중 선택	week5

캐글 산탄데르 고객 만족 예측

1. 타겟 데이터: 불균형한 데이터→ 데이터 분리 시 소수의 데이터가 테스트, 학습 데이터에 잘 나뉘어 들어갔는지 확인 필요
2. kfold를 이용하며 xgb모델을 베이지안 최적화하는 흐름
   a. **목적**: 어떤 파라미터 세트로 학습해야 auc가 높게 나오는가?
   b. **주의**: 파라미터 세트가 같더라도 각각의 kfold를 수행하면서 달라지는 학습 세트에 따라 xgb의 결정트리 모양과 가중치는 달라짐 -> 파라미터와 결정트리 모양 및 가중치 혼동하지 않기
   c. **전체 흐름**

      베이지안 (fmin)
      → 파라미터 세트 제안
      예: max_depth=7, learning_rate=0.05

      → KFold 3번 반복 (신뢰있는 AUC 측정을 위해)

      → XGB 학습 (3번 모두 베이지안이 제안한 같은 파라미터로 다른 학습 데이터를 통해 트리 쌓으며 학습)
      → AUC 측정

      → 3번 AUC 평균 → 베이지안에 반환

      → 베이지안: "이 파라미터는 0.85였네, 다음엔 이쪽을 탐색하자"
      → 다음 파라미터 세트 제안
      → 반복...
   d. **xgb의 흐름**

      n_estimators만큼 결정 트리를 하나 씩 추가해가며 학습함
      → 이때 추가한 결정 트리는 앞의 결정 트리의 오차를 학습함
      ex) 앞의 결정트리 예측값: 400, 실제값: 500 -> 뒤의 결정트리는 500-400=100인 **앞의 결정트리들의 오차를 학습**

Decision Tree And Random Forest Classifier Models

1. `dict()`: 딕셔너리 만드는 코드, plotly이나 matplotlib 같은 시각화 lib 사용 시 자주 쓰임
2. 카데고리형 데이터 정수로 변경하는 법
   a. 여기에서는 **조건문 사용**

      +) for문으로 값을 바꿔도 pandas가 컬럼 타입을 자동으로 바꿔주지 않음
      → object 타입 컬럼에 숫자를 넣어도 object 유지 됨 → 명시적으로 타입을 바꿔줘야 함

      ```
      for i in range(0,len(dataclass.BP)):
          if dataclass.BP[i] == "LOW":
              dataclass.BP[i] = 2
          elif dataclass.BP[i] == "NORMAL":
              dataclass.BP[i] = 1
          else:
              dataclass.BP[i] = 0
      ```

   b. but **map()을 이용**하는게 더 가독성이 좋고 짧음

      ```
      dataclass['BP'] = dataclass['BP'].map({'LOW': 2, 'NOR
      MAL': 1, 'HIGH': 0})
      ```

   c. **LadelEnder**도 사용 가능하지만 원하는 라벨로 지정할 경우 좀 번거롭
3. 멀티 레이블일때 confusion matrix 보는 법
   a. One vs Rest 방식으로 해석
   b. ex) 0번 클래스 기준
      - TP: 행렬의 대각선 원소
      - FN: 대각선 원소를 제외한 0번 행 원소
      - FP: 대각선 원소를 제외한 0번 열 원소
      - TN: 나머지 전부

Beginner Friendly CATBOOST with OPTUNA

1. `df.duplicated().sum()`
   - 중복된 행이 몇 개인지 세는 코드

- 데이터 전처리할 때 중복 행 있으면 모델 학습에 영향 줄 수 있어서 초반에 꼭 확인

2. `df[categorical].nunique()`
   - unique() 는 고유값을 직접 보여줌
   - nunique() 는 고유값의 개수만 보여줌
3. zero varianc: 고유값이 1개, 쓸모없는 칼럼
4. extremely high variance: 고유값이 너무 많음, 수치형이 될 위험 O
5. skewness 확인
   - 0: 데이터가 좌우 대칭이면 왜도 = 0
   - 양수: 오른쪽으로 꼬리가 긴 분포
   - 음수: 왼쪽으로 꼬리가 긴 분포
6. Baseline Modle: 실제 학습을 하는 게 아니라 무조건 1만 예측하는 멍청한 모델
   -> 비교 기준점(베이스라인)을 만들기 위해
   => 이후에 만드는 진짜 모델들이 이것보다는 높아야 "의미있는 모델" 이라고 할 수 있기 때문
7. make_pipeline() 함수: 인코딩 학습 예측을 편하게 할 수 있음
   a. 원래

```
ct.fit_transform(X_train) # 인코딩

model.fit(X_train, y_train) # 학습

ct.transform(X_test) # 테스트 인코딩

model.predict(X_test) # 예측
```

   b. make_pipeline() 사용

```
pythonpipe = make_pipeline(ct, model)

pipe.fit(X_train, y_train) # 인코딩 + 학습 한번에

pipe.predict(X_test) # 인코딩 + 예측 한번에
```

8. 앙상블 종류
    - AdaBoostClassifier → 틀린 샘플에 가중치를 높여가며 학습
    - GradientBoostingClassifier → 이전 트리의 오차를 보완하며 학습
    - RandomForestClassifier → 여러 트리를 병렬로 만들어 다수결
    - **ExtraTreesClassifier → 랜덤포레스트랑 비슷한데 분기점을 완전 랜덤으로 정함**
9. RandomForest vs ExtraTree
    - RandomForest → 분기점을 최적의 값으로 찾음 (50이 제일 좋다고 계산)
    - ExtraTree → 분기점을 그냥 랜덤으로 정함 (그냥 50)
        - ExtraTree가 더 빠른 but 랜덤이라 성능이 들쭉날쭉할 수 있음

10. CATBOOST: **Cat**egorical **Boost**ing의 줄임말로, 러시아 회사 Yandex에서 만든 그래디언트 부스팅 계열 모델
    - 다른 모델과의 차이점
        - 랜덤포레스트: 트리를 **병렬로** 여러 개 생성
        - CatBoost: 트리를 **순서대로** 생성하며 이전 트리의 오차를 다음 트리가 보완
    - 특징
        - 범주형 변수를 **자동으로 처리** (`cat_features` 만 지정하면 됨)
        - 별도의 Label Encoding, One-Hot Encoding 불필요
    - 기본 목적함수 자동 선택
        - 타겟이 2개 (0, 1) → **Logloss**
        - 타겟이 3개 이상 → **MultiClass**
11. Optuna vs Hyperopt
    - 탐색 방식
        - Hyperopt → TPE(Tree-structured Parzen Estimator) 방식만 지원
        - Optuna → TPE 포함 여러 방식 지원하고 자동으로 골라줌
    - 사용 편의성
        - Hyperopt → 탐색 공간을 따로 딕셔너리로 정의해야 해서 코드가 복잡

    - Optuna → trial.suggest_int, trial.suggest_float 이렇게 함수 안에서 바로 정의해서 직관적
- 시각화
    - Hyperopt → 시각화 기능 거의 없음
    - Optuna → 자체 시각화 기능이 있어서 trial 결과를 그래프로 볼 수 있음
- 결론: 요즘은 Optuna를 더 많이 씀 -> 코드도 깔끔하고 기능도 많기 때문